

entregable_2_exo_condensado_filtered

May 29, 2026

1 Modelos estocásticos y redes neuronales

Materia: Análisis de Series de Tiempo y Pronósticos (1C-2026)

Grupo: 9

Integrantes:

- Lucas Achaval - Email: lachavalrodriguez@estudiantes.unsam.edu.ar
- Marcos Achaval - Email: machavalrodriguez@unsam-bue.edu.ar

Título del entregable: Modelos estocásticos y redes neuronales

Conjunto de datos original: dataset propio de una estación meteorológica en un club náutico de Potrerillos (disponibles en [Windguru](#)).

1.1 Resumen

Este trabajo busca construir un modelo de pronóstico horario de viento promedio (`wind_avg`) en Potrerillos, Mendoza, con horizonte de 12 horas, evaluado mediante MAE, RMSE y R^2 . En la primera entrega caracterizamos la serie (ciclo diurno marcado, estacionariedad confirmada por ADF, picos de ACF/PACF en lags 1-2 y 20-24) y entrenamos un LassoCV con 24 lags, 6 exógenas y codificación cíclica de la hora, que redujo el RMSE **aproximadamente un 25 %** y elevó el R^2 de 0.117 a 0.498 frente a una persistencia estacional, sin signos de sobreajuste. Esta entrega avanza con benchmark, modelos estocásticos y redes neuronales.

1.2 Carga y preprocesamiento

Replicamos el preprocesamiento del primer entregable: lectura del archivo crudo, remuestreo horario (mediana para variables lineales, media circular para `wind_direction`) y agregado del atributo auxiliar `interval`. La variable `wind_min` queda fuera porque está completamente vacía, y `gustiness` se omite porque es una derivada de `wind_avg` y `wind_max`.

1.2.1 Split de entrenamiento y test

Separamos un **80 % inicial cronológico** como entrenamiento y el **20 % final** como test. Toda la búsqueda de hiperparámetros (orden SARIMA, arquitectura NN) y los diagnósticos se hacen solo sobre el train; el test se reserva para el desempeño final de todos los modelos **bajo la misma metodología de evaluación**.

Los 276 faltantes (4.5 %) se completan con `ffill().bfill()`. Definimos las constantes globales `HORIZONTE = 12` (pasos a predecir) y `PERIODO = 24` (estacionalidad horaria).

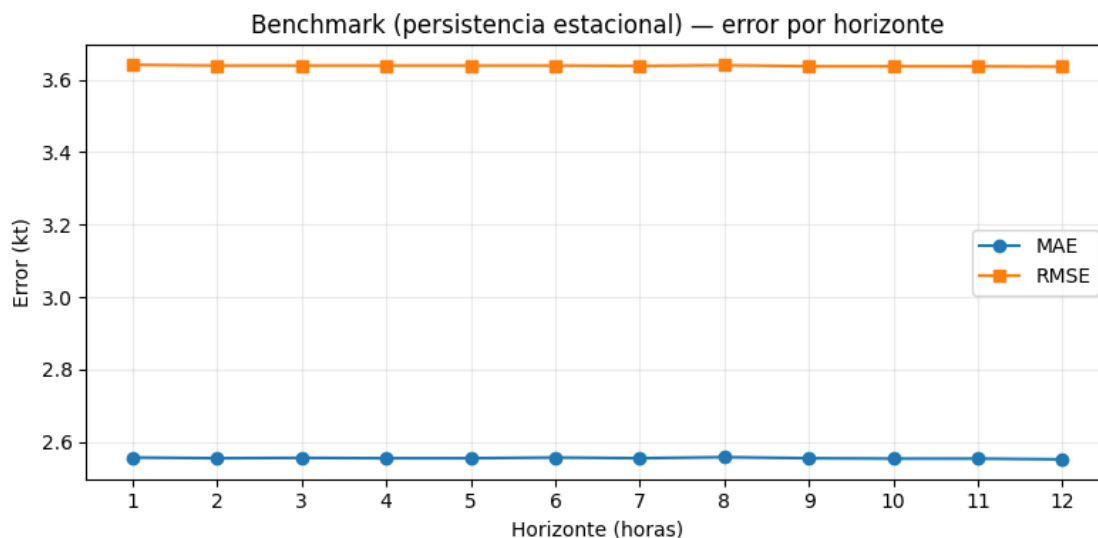
1.3 Modelo de referencia (benchmark)

Como benchmark adoptamos la **persistencia estacional** con período 24 h. Para un instante t y un horizonte $h \leq 24$:

$$\hat{y}_{t+h} = y_{t+h-24}$$

Cada hora se pronostica con el valor observado a la misma hora del día anterior. Es simple, sin parámetros a estimar, y aprovecha la única estructura fuerte de la serie: el pico del ACF en lag 24 (≈ 0.5) detectado en el entregable 1. Una persistencia simple ($\hat{y}_{t+h} = y_t$) ignoraría esa estacionalidad y rendiría peor. Fija una línea de base no trivial que los modelos más complejos deben superar.

Lo evaluamos sobre `serie_test` con un esquema **walk-forward**: para cada origen t producimos pronósticos a 1...12 h y los contrastamos con las observaciones. Es la misma metodología que aplicaremos a SARIMA y a las redes, de modo que las tablas son comparables.



1.3.1 Desempeño del benchmark

MAE, RMSE y R^2 del benchmark por horizonte (1 a 12 h):

- **Error casi constante con el horizonte.** Como la predicción siempre usa un dato de 24 h atrás, da igual cuán lejos esté el target dentro de las próximas 24 h. Este perfil plano contrasta con los modelos con memoria de corto plazo (SARIMA, NN), que se degradan al alejarse el horizonte.
- **R^2 positivo pero bajo.** La estacionalidad diaria sola explica una fracción modesta de la varianza, dejando margen para que los modelos posteriores aporten vía lags cercanos y exógenas.

Este es el desempeño mínimo que deben superar los modelos siguientes.

1.4 Modelo estocástico

A partir de la ACF y PACF buscamos un modelo estocástico para `wind_avg`. El entregable 1 ya mostró tres hechos clave:

- ADF sobre la serie original: estadístico -11.4 , p -valor $7.6 \times 10^{-21} \rightarrow$ **la serie es estacionaria**.
- ACF con pico claro en lag 24; PACF significativa en lags 1-2 y 20-24.
- Espectro dominado por 1 ciclo/día, con un segundo pico (más débil) en 2 ciclos/día.

1.4.1 ¿Por qué SARIMA y no AR, MA, ARMA o ARIMA?

La elección se desprende de lo que muestra la serie. Las reglas (guías 4 y 5):

Modelo	ACF esperado	PACF esperado
AR(p)	decae	corta en lag p
MA(q)	corta en lag q	decae
ARMA(p, q)	decae	decae

- **AR puro:** descartado. La PACF no corta limpio (clusters en 1-2 y 20-24); capturar el pico de lag 24 exigiría $p = 24$, contra la parsimonia.
- **MA puro:** descartado. La ACF no corta en lags cortos: decae con oscilaciones (lag 12, pico en lag 24).
- **ARMA:** descartado. No incorpora la noción de **período**; el pico aislado en lag 24 es estacional y exigiría muchos coeficientes consecutivos.
- **ARIMA:** descartado. El ADF confirma estacionariedad ($d = 0$), así que ARIMA($p,0,q$) colapsa al ARMA ya descartado.
- **SARIMA:** el ajuste natural. Modela la estacionalidad con un solo término P o Q sobre el rezago $\mathbf{B}^{24}$ (guía 5):

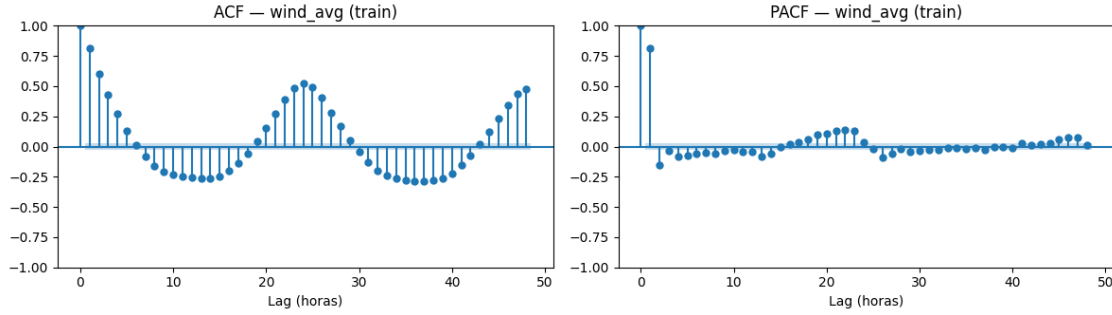
$$\Theta_P(\mathbf{B}^{24}) \Theta_p(\mathbf{B}) (1 - \mathbf{B}^{24})^D (1 - \mathbf{B})^d x_t = \Phi_Q(\mathbf{B}^{24}) \Phi_q(\mathbf{B}) w_t$$

Un único coeficiente estacional reemplaza decenas de no estacionales. Tomamos $d = 0$ (ADF) y $D = 0$ (el pico estacional ya está presente sin diferenciar); si los residuos muestran estructura cerca de múltiplos de 24, reconsideramos $D = 1$.

Exploramos entonces un **SARIMA(p,0,q)(P,0,Q,24)**, dejando p, q, P, Q a la búsqueda por AIC sobre `serie_train`.

1.4.2 Funciones de autocorrelación sobre el training

Repetimos el análisis del entregable 1 **solo sobre el training**, para no mirar el test al elegir hiperparámetros: toda inspección que defina el modelo debe basarse en datos vistos.



El comportamiento sobre el training reproduce el entregable 1: PACF significativa en lags 1-2 y 20-24, ACF con decaimiento lento y pico en lag 24. Esto confirma SARIMA y define la grilla:

- $p \in \{0, 1, 2\}$ (PACF hasta lag 2)
- $q \in \{0, 1\}$ (MA corto opcional)
- $P, Q \in \{0, 1\}$ (un término estacional alcanza para el pico de lag 24)
- $d = D = 0, s = 24, \text{trend} = 'c'$

Son $3 \times 2 \times 2 \times 2 = 24$ combinaciones, en línea con la **parsimonia**.

1.4.3 Selección del orden con el criterio de Akaike (AIC)

Ajustamos un SARIMA por combinación con `statsmodels.tsa.arima.model.ARIMA` y registramos el AIC:

$$\text{AIC} = -2 \log \mathcal{L} + 2k$$

con $\mathcal{L}$ la verosimilitud y k el número de parámetros. Penaliza la cantidad de parámetros, por lo que minimizarlo selecciona el modelo más simple compatible con los datos. Envolvemos el ajuste en `try/except` para descartar combinaciones que no converjan.

La tabla ordena las 24 combinaciones por AIC creciente. Tomamos como ganador la primera fila (AIC mínimo).

1.4.4 Ajuste del modelo ganador

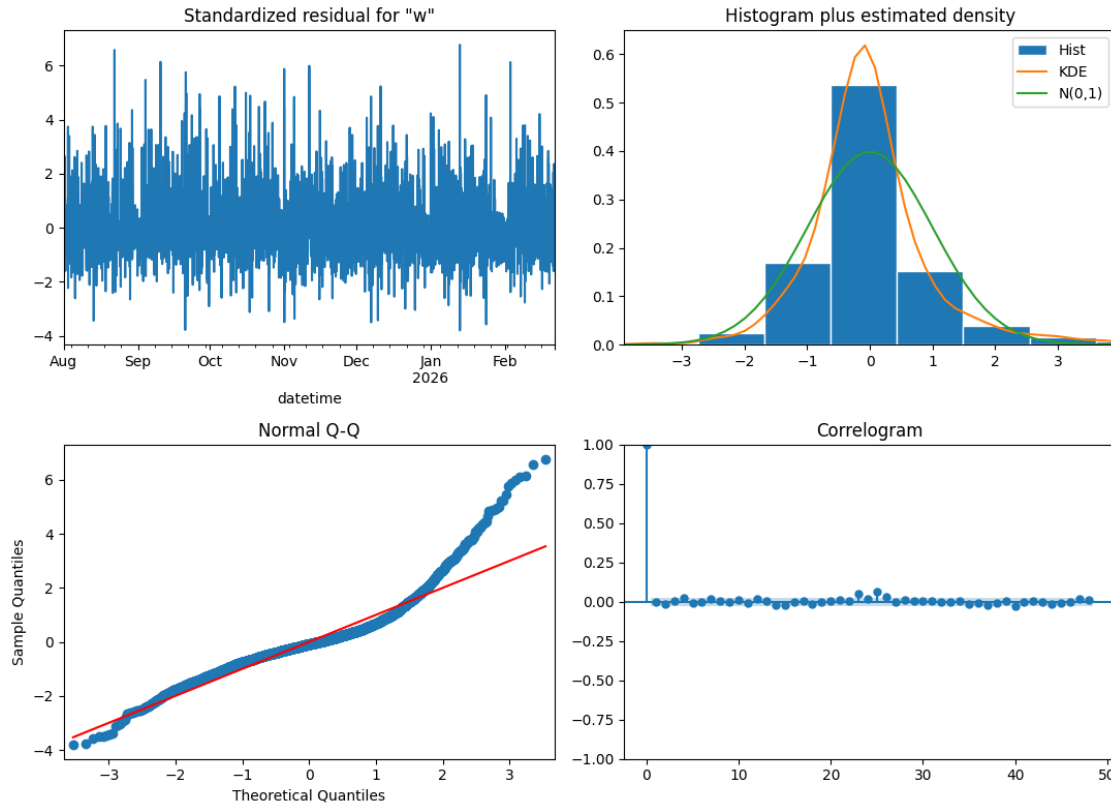
Refitteamos el orden ganador y revisamos `summary()`: nos interesa que los términos sean significativos ($P > |z|$ cerca de 0) y que las raíces caigan fuera del círculo unitario (estacionariedad e invertibilidad).

1.4.5 Diagnóstico de residuos

Si el modelo captura toda la estructura, los residuos deben comportarse como ruido blanco (independientes, idénticamente distribuidos, media cero). `plot_diagnostics()` agrupa:

1. **Residuos en el tiempo:** sin tendencia, heterocedasticidad ni outliers sistemáticos.
2. **Histograma + KDE vs $\mathcal{N}(0, 1)$:** ajuste razonable a la normal.
3. **Q-Q plot:** puntos sobre la diagonal; desvíos en colas indican cola pesada.

4. **Correlograma:** todos los lags dentro de la banda, sin estructura sobreviviente.



Interpretación del diagnóstico:

- **Residuos en el tiempo:** oscilan alrededor de 0 sin tendencia ni cambio de varianza. Hay picos aislados (magnitud 4-7) repartidos, sin clusters que sugieran heterocedasticidad fuerte.
- **Histograma + KDE vs $N(0,1)$:** la distribución empírica no coincide con la normal: más masa cerca de 0 y en las colas, menos en la zona intermedia. La varianza total se mantiene cerca de 1, pero la forma no es gaussiana.
- **Q-Q plot:** alineado en el centro y la cola izquierda; en la cola derecha (desde +1.5) los cuantiles muestrales se elevan muy por encima de la diagonal (llegan a 6-7 vs 3 teóricos). Indica **asimetría positiva**: el modelo predice bien vientos bajos/normales pero subestima los picos altos. Tiene sentido físico: el viento no puede ser muy negativo pero sí subir mucho.
- **Correlograma:** casi todos los lags 1-48 caen dentro de la banda del 95 %. Un par (23 y 25) asoman apenas; con 48 lags es esperable por azar y no justifican complicar el modelo.

El SARIMA absorbió la estructura temporal y la selección de (p, q, P, Q) por AIC queda validada. La condición crítica de ruido blanco (autocorrelación nula) se cumple. La no-normalidad de los residuos es esperable en series de viento (eventos extremos) y no invalida los coeficientes, pero los intervalos de predicción basados en la hipótesis gaussiana subestimarán los eventos extremos.

En la próxima sección evaluamos el modelo sobre `serie_test` con un walk-forward idéntico al del benchmark.

1.5 Evaluación del modelo estocástico

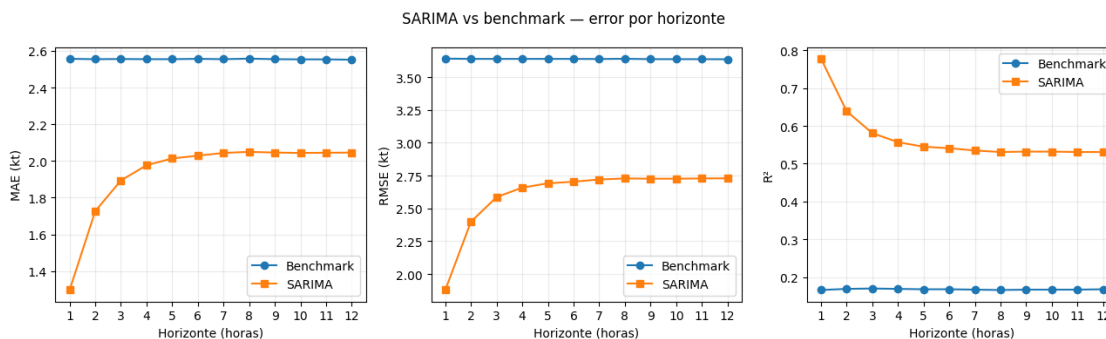
1.5.1 Metodología (windowing)

Ajustamos SARIMA una vez sobre `serie_train` y reportamos métricas sobre `serie_test`. La guía 5 usa `extend()` para evaluar a 1 paso; como la consigna pide métricas por horizonte (hasta $h = 12$), lo extendemos de forma iterativa con un **walk-forward** origen por origen:

1. Para cada origen t producimos un pronóstico a 12 h con `forecast(steps=12)` y registramos $\hat{y}_{t+1}, \dots, \hat{y}_{t+12}$.
2. Antes de avanzar, incorporamos la observación con `append(refit=False)`, **sin re-estimar coeficientes**, dejando el filtro de Kalman al día con el último dato, igual que en operación.

¿Por qué no re-fitear en cada origen? Re-estimar en cada uno de los ~1200 orígenes implicaría ajustar SARIMA estacional miles de veces, con costo enorme y mejora marginal. Lo habitual es entrenar al inicio y solo actualizar el estado.

Calculamos MAE, RMSE y R^2 por horizonte con la misma función `metricas_por_horizonte` del benchmark, así ambas tablas quedan comparables.



1.5.2 Interpretación de los resultados

- **SARIMA supera al benchmark en todos los horizontes.** Las curvas no se cruzan; incluso en el horizonte más largo el RMSE queda por debajo y el R^2 es varias veces mayor.
- **Degradación en los primeros pasos y meseta desde $h = 6$.** El bloque $AR(2)/MA(1)$ decae rápido (~5-6 pasos), mientras el bloque estacional $(1, 0, 1, 24)$ (con `ar.S.L24` cercano a 1) se comporta como un ciclo estable. A horizontes largos la predicción converge a la media condicional más el ciclo diario.
- **La meseta queda por encima del benchmark.** A horizontes lejanos ambos usan el ciclo de 24 h, pero SARIMA trabaja con una estimación suavizada y el benchmark copia un único dato ruidoso de ayer.

La línea de base sube respecto al benchmark: cualquier red deberá superar estas cifras.

1.6 Modelo de redes neuronales

Armamos redes neuronales siguiendo la guía 7 (LSTM endógeno). A diferencia de SARIMA, una red no asume un proceso lineal y aprende una función cualquiera desde una ventana de pasos pasados hacia los 12 futuros. Eso obliga a dos pasos de preprocesamiento:

- **Normalización** de la entrada, para que el optimizador trabaje en escala controlada.
- **Ventaneo**, para transformar la serie en pares (X_i, y_i) que la red consuma como muestras.

Mantenemos el split 80/20, el horizonte de 12 h y la frecuencia horaria. Probamos **tres configuraciones**: una densa de referencia, una LSTM simple con dropout y una LSTM apilada con `return_sequences=True`.

1.6.1 Normalización y ventaneo

Normalización Z-score con estadísticos del train. Restamos media y desvío calculados **solo sobre `serie_train`** y aplicamos la misma transformación a `serie_test`, evitando filtrar el test y dejando la entrada centrada en 0 con varianza 1. Casteamos a `float32` (dtype por defecto de Keras).

Ventaneo sequence-to-vector con salida multi-paso. Para cada origen i :

- $X_i = (x_i, x_{i+1}, \dots, x_{i+W-1})$ — los W valores pasados,
- $y_i = (x_{i+W}, x_{i+W+1}, \dots, x_{i+W+H-1})$ — los $H = 12$ valores futuros.

Tomamos $W = 48$ para meter **dos ciclos diarios completos** en la ventana (guía 7). La última capa densa tiene 12 unidades y predice los 12 horizontes en un solo paso (*direct multi-step*, ejercicio 7): más simple que el esquema recursivo, evita el error acumulado y deja la comparación con SARIMA más prolija.

1.6.2 Arquitecturas propuestas

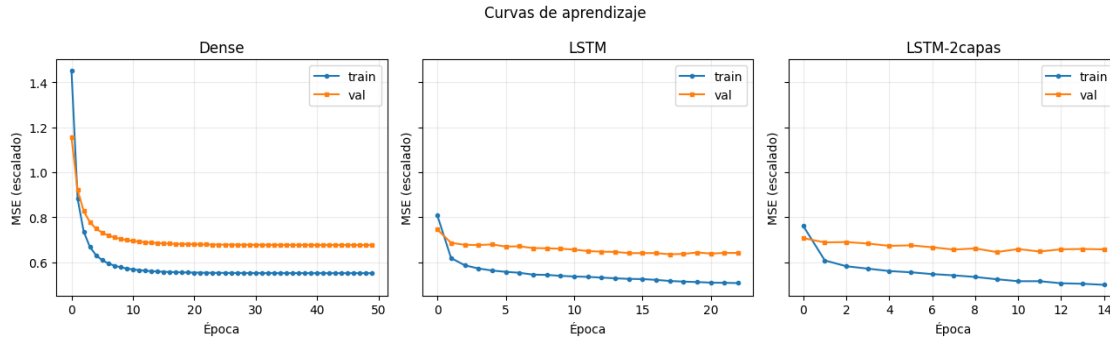
Las tres comparten entrada (`Input((48, 1))`) y salida (`Dense(12, activation='linear')`), y difieren en el medio:

- **Dense:** `Flatten` aplana la ventana a 48 valores y un `Dense(12)` aprende coeficientes lineales hacia cada horizonte. Es una regresión lineal multi-objetivo sobre los lags; sirve de referencia para medir el aporte de la estructura recurrente.
- **LSTM:** `LSTM(32)` recorre la ventana y entrega el estado oculto final, seguido de `Dropout(0.2)` (desactiva al azar el 20 % de las unidades para regularizar) y `Dense(12)`. Arquitectura típica de la guía 7.
- **LSTM-2capas:** apila dos LSTM; la primera con `return_sequences=True` entrega la secuencia completa de estados a la segunda (ejercicio 5), cada una con `Dropout(0.2)`. Evalúa si más profundidad ayuda.

Optimizador Adam, pérdida MSE (guía 7). Semilla de Keras 99 para reproducibilidad.

1.6.3 Entrenamiento

Entrenamos cada modelo sobre `X_train_nn` con `validation_split=0.1` (último 10 % del train, en orden temporal) y `EarlyStopping(patience=5)` sobre la pérdida de validación, restaurando los mejores pesos (ejercicio 4, guía 7). `epochs=50` como tope.



1.6.4 Resumen del ajuste

Modelo	Parámetros	Épocas ajustadas	val_loss mínima
Dense	588	50 (tope)	0.676
LSTM	4 748	23	0.635
LSTM-2capas	13 068	15	0.645

- **Las dos LSTM dispararon EarlyStopping antes del tope** (23 y 15 épocas). La Dense llegó al tope de 50 sin dispararlo, con su pérdida bajando despacio, así que su 0.676 es una cota superior.
- **La LSTM simple obtiene la menor val_loss** (0.635). La diferencia con la apilada (0.010) sale de una sola semilla y puede ser ruido; lo afirmable es que **ambas LSTM quedan por debajo de la Dense**, y entre LSTM y LSTM-2capas la diferencia no es concluyente.
- **Train y validación divergen poco**: el dropout = 0.2 regulariza sin sobreajuste agresivo.

Las pérdidas están en **escala normalizada** y promediadas sobre los 12 horizontes, no comparables con el RMSE en kt. La sección siguiente aplica el mismo walk-forward que SARIMA para reportar en kt.

1.7 Evaluación de la red neuronal

1.7.1 Modelo elegido y metodología

Elegimos la **LSTM simple**: menor val_loss (0.635) y, frente a la de dos capas, con menos de la mitad de parámetros (4 748 vs 13 068). Por parsimonia, la más chica.

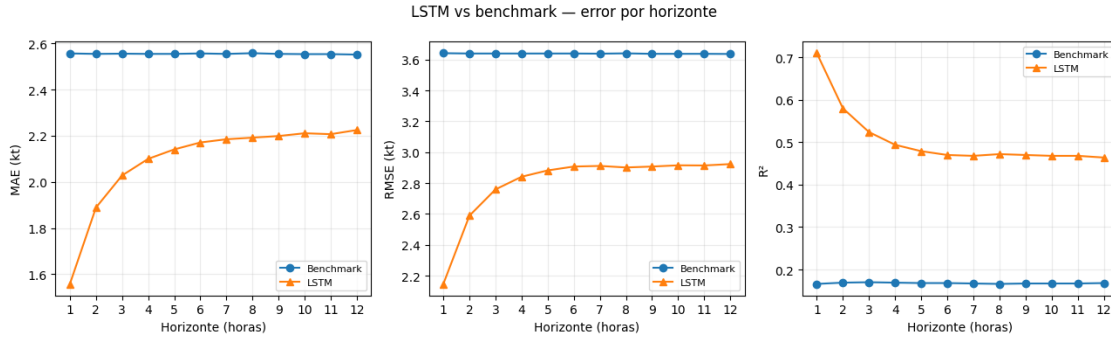
La evaluamos sobre `serie_test` con la **misma metodología que SARIMA**: walk-forward con MAE, RMSE y R^2 por horizonte (1 a 12 h) en kt, contra el benchmark. La comparación con SARIMA y la elección operativa quedan para el punto 8.

El formato es *direct multi-step*: una pasada de `predict()` sobre `X_test_nn` devuelve los 12 horizontes. Cada fila es un origen cuya predicción usa solo los 48 valores previos, equivalente al loop `forecast → append(refit=False)` de SARIMA.

Dos detalles:

- **Desnormalización antes de medir**: recuperamos kt con `y * train_std + train_mean`.

- **Offset respecto de SARIMA:** el ventaneo arranca en `serie_test.iloc[WINDOW_SIZE + h - 1]` vs `iloc[h - 1]` de SARIMA: la LSTM se evalúa sobre 1170 orígenes y SARIMA sobre 1218. La diferencia es chica y los promedios siguen comparables.



1.7.2 Interpretación de los resultados

- **La LSTM supera al benchmark en todos los horizontes.** En $h = 1$ el salto es grande: RMSE 2.15 kt vs 3.64 y R^2 0.71 vs 0.17. La red aprovecha los lags cercanos de la ventana de 48 h, algo que la persistencia estacional (un único dato de 24 h atrás) no puede.
- **Degradación en los primeros pasos y meseta desde $h = 6$.** El RMSE crece de 2.15 a ~2.9 kt entre $h = 1$ y $h = 6$ (R^2 baja de 0.71 a ~0.46). Mismo perfil que SARIMA: la memoria corta aporta al inicio y a horizonte lejano converge al ciclo diario.
- **La meseta queda por debajo del benchmark.** En $h = 12$ la LSTM (RMSE 2.92 kt, R^2 0.46) le gana a la persistencia (RMSE 3.64, R^2 0.17).

La LSTM supera con claridad la línea de base. En el punto 8 la comparamos contra SARIMA.

1.7.3 ¿Los coeficientes son interpretables como importancia de atributos?

No de forma directa. Las redes reúnen entre 588 (Dense) y 13 068 (LSTM-2capas) pesos, y la relación peso importancia no es la de una regresión lineal.

- **Dense** es el único caso donde los 588 pesos del `Dense(48 → 12)` mapean *literalmente* $\{\text{lag de entrada}\} \times \{\text{horizonte}\}$. Aun así la magnitud no equivale a importancia: la entrada está normalizada y los 48 lags están **fuertemente correlacionados** (lags 1-2 y 24), lo que reparte el crédito entre lags equivalentes y vuelve engañosos los pesos individuales.
- **LSTM y LSTM-2capas** no admiten esa lectura. Los pesos viven en matrices de forma $(d_{\text{in}} + d_h) \times 4d_h$ que mezclan no linealmente input, estado oculto y los cuatro *gates* (input, forget, cell, output) a lo largo de los 48 pasos. No hay correspondencia uno-a-uno peso lag; la importancia depende de toda la trayectoria de estados ocultos.

1.8 Redes neuronales con variables exógenas

Incorporamos variables exógenas siguiendo la guía 8 (*LSTM-exo*): cada paso de la ventana pasa de escalar a vector con el target más los atributos meteorológicos, de modo que la entrada tiene forma $(48, \text{n_features})$ en lugar de $(48, 1)$. La LSTM ve la evolución conjunta de todas las variables.

Atributos. Las mismas cinco exógenas que el Lasso (wind_max, temperature, rh, mslp, wind_direction). Así la LSTM-exo queda comparable contra la **LSTM univariante** (misma arquitectura, sin exógenas) y el **Lasso** (mismos atributos, modelo lineal). La pregunta es la del Lasso: *¿las exógenas aportan algo, o wind_avg ya contiene casi toda la señal predecible?*

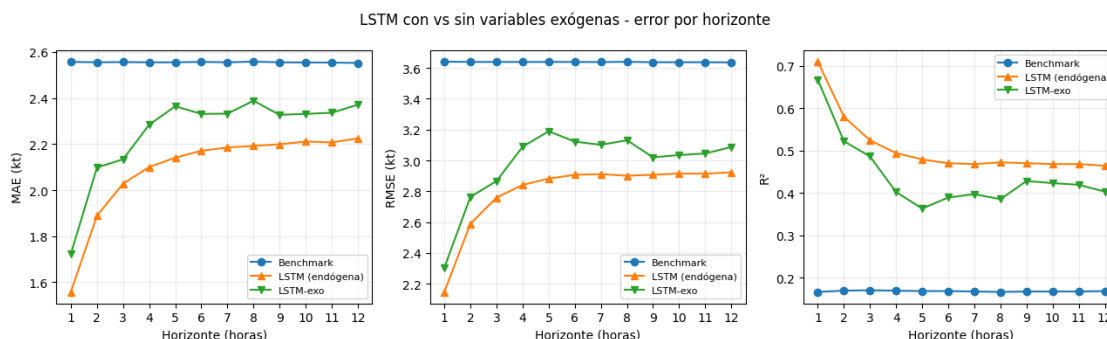
Decisiones de diseño (mínimas, para aislar el efecto de las exógenas):

- **Reutilizamos la arquitectura ganadora** (LSTM(32) → Dropout(0.2) → Dense(12)) y cambiamos solo el ancho de entrada. Cualquier diferencia es atribuible a las exógenas.
- **Solo valores pasados** dentro de la ventana: en el origen la red ve las últimas 48 h de las seis variables y proyecta 12 pasos de wind_avg. Mismo conjunto de información que el Lasso, sin fuga de datos.
- **Z-score por atributo** con estadísticos del train; desnormalizamos la salida con la media y el desvío de wind_avg antes de medir.
- *Salvedad:* wind_direction es circular (0°/360° son el mismo punto) y el Z-score no lo respeta; lo dejamos crudo por consistencia con el Lasso. Codificarla como seno/coseno sería la mejora natural.

Ventanas y arquitectura. ventanas_mv arma, para cada origen, una ventana (48, 6) con las seis variables y un target (12,) con **solo** los 12 valores futuros de wind_avg (columna 0). build_lstm_exo es idéntica a la LSTM ganadora salvo Input((48, 6)). Mismo protocolo de entrenamiento: validation_split=0.1, EarlyStopping(patience=5) con restauración de los mejores pesos, semilla 99.

1.8.1 Evaluación de la LSTM-exo

Misma metodología que la LSTM univariante: una pasada de predict() sobre el test, desnormalización a kt y métricas por horizonte (1 a 12 h). Como comparte ventana ($W = 48$) y horizonte ($H = 12$) con la LSTM endógena, ambas se miden sobre los mismos orígenes y son directamente comparables.



1.8.2 Interpretación de los resultados

- **Las exógenas mejoran el ajuste pero empeoran la generalización.** En entrenamiento la LSTM-exo logró menor val_loss (0.567 vs 0.635), pero **sobre el test el orden se invierte**: es peor en todos los horizontes (RMSE promedio 2.98 vs 2.80 kt, R^2 0.44 vs 0.51), perdiendo ya en $h = 1$ (2.31 vs 2.15 kt).

- **Es un caso de libro de sobreajuste.** Las exógenas suman grados de libertad que la red usa para memorizar relaciones del train/validación que no se sostienen en el test. `wind_max` correlaciona fuerte con el target, pero `temperature`, `rh`, `mslp` y `wind_direction` aportan sobre todo ruido a esta resolución horaria.
- **Coincide con el Lasso** (punto 8), que tampoco saca ventaja de las exógenas. La señal predecible de `wind_avg` vive casi por completo en su estructura autorregresiva y estacional.

Lección metodológica: más variables de entrada no implican mejor pronóstico. La `val_loss` más baja podría engañar; es la **evaluación honesta sobre el test** la que revela que las exógenas no ayudan. Para el punto 8 conservamos la **LSTM univariante** como representante de las redes.

1.9 Comparación de modelos y elección

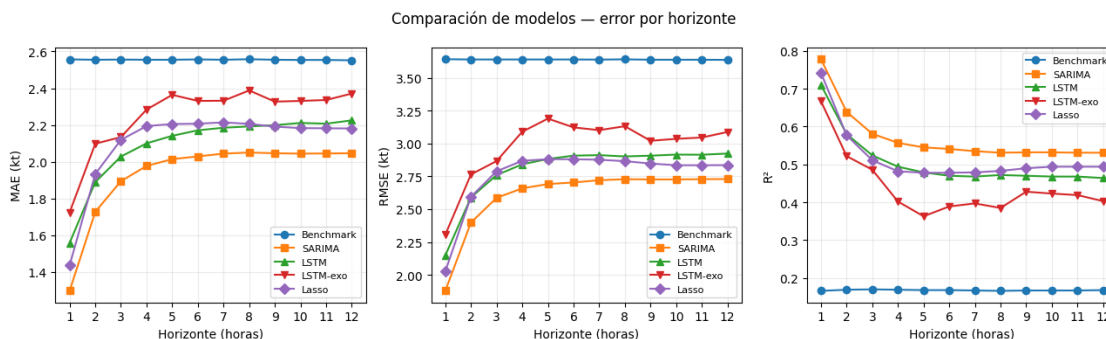
Reunimos los cinco modelos bajo la **misma metodología** (walk-forward por horizonte sobre el 20 % final, métricas en kt):

- **Benchmark:** persistencia estacional (punto 3),
- **SARIMA**(2,0,1)(1,0,1,24) (punto 4),
- **LSTM** simple univariante (punto 6),
- **LSTM-exo** con cinco variables exógenas (punto 6),
- **LassoCV** del primer entregable.

1.9.1 Re-evaluación del LassoCV bajo el mismo esquema

El LassoCV del entregable 1 se evaluó con validación cruzada sobre todo el dataset, no comparable con este walk-forward. Re-entrenamos el mismo pipeline (`StandardScaler` → `LassoCV` con `TimeSeriesSplit` para elegir α) sobre el 80 % inicial y lo evaluamos sobre el 20 % final, horizonte por horizonte.

- **Es uno de los dos modelos con exógenas** (junto a la LSTM-exo): 24 lags de `wind_avg` más las cinco exógenas y la hora cíclica, todas conocidas en el origen. Benchmark, SARIMA y LSTM simple son univariantes. La comparación responde: **¿la información exógena justifica su costo frente a los modelos univariantes?**
- **Direct multi-step:** un modelo por horizonte (`objetivo = wind_avg.shift(-h)`), con un `gap` de 12 h entre train y test para no filtrar.
- **Conteos de orígenes algo distintos** (1218 benchmark/SARIMA, 1170 las dos LSTM, 1217-1228 el Lasso según el horizonte); los sub-períodos se solapan casi por completo, así que los promedios siguen comparables.



1.9.2 Interpretación y elección del modelo

Resumen (medias sobre los 12 horizontes):

- **Los cuatro modelos superan al benchmark en todos los horizontes.** La persistencia (RMSE 3.64 kt, R^2 0.17, plana) queda muy por debajo; las mejoras de RMSE promedio van del 18 % (LSTM-exo) al 28 % (SARIMA).
- **SARIMA es el mejor en todo el rango** (RMSE promedio 2.61 kt, R^2 0.57), con ventaja máxima en el corto plazo ($h = 1$: R^2 0.78 vs 0.74 Lasso, 0.71 LSTM, 0.67 LSTM-exo) y manteniéndose arriba hasta $h = 12$.
- **Los dos modelos con exógenas no se despegan**, y la **LSTM-exo es la peor** (2.98 kt, R^2 0.44). Lasso (2.76 kt) y LSTM univariante (2.80) quedan empatados por debajo de SARIMA. Hallazgo central, por partida doble: **ni el modelo lineal ni la red convierten las cinco exógenas (wind_max, temperatura, humedad, presión, dirección) en mejor pronóstico**; en la red incluso son contraproducentes (bajan la `val_loss` pero suben el error de test). La estructura autorregresiva y estacional de `wind_avg` ya contiene casi toda la señal predecible.
- **A horizontes largos los modelos convergen** (~2.7–2.9 kt de RMSE en $h = 12$): todos se apoyan en el ciclo diario y las diferencias se achican.

¿Qué modelo elegiríamos para una fase operativa? **SARIMA.**

1. **Mejor desempeño en todos los horizontes**, con la mayor ventaja justo donde más se usa un pronóstico de viento: las próximas horas.
2. **Robustez operativa.** Es univariante: solo necesita el histórico de `wind_avg`. No depende de cinco sensores exógenos que pueden fallar o llegar tarde y, de hecho, los modelos que sí los usan (Lasso y LSTM-exo) no logran sacarles ventaja.
3. **Parsimonia e interpretabilidad.** Pocos parámetros, ajuste rápido (re-fitteable de forma periódica) e intervalos de predicción nativos, con la salvedad del punto 4: los residuos no son gaussianos, así que esos intervalos subestiman los eventos extremos.
4. **Las redes neuronales, las más complejas** (entrenamiento, sensibilidad a la semilla, menor interpretabilidad), no compensan: la univariante empata al Lasso y queda bajo SARIMA, y agregarle exógenas la empeora.

Para esta serie y este horizonte, el modelo estocástico ofrece la mejor relación desempeño/simplicidad/robustez.